INDEX

Practical No.	Topic	Date	Signature
1.	Environment Setup and JavaScript Basics a. Install Node.js and Visual Studio Code, and verify installation. b. Create a simple Node.js file and run it using the terminal. c. Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations). d. Create and run a simple program using variables and functions. e. Demonstrate use of operators and control statements in JavaScript f. Write programs using loops and template literals.		
2.	Core JavaScript Concepts for MERN a. Create functions and call them with parameters. b. Work with arrays and objects in JavaScript. c. Demonstrate use of arrow functions. d. Write a program using array methods (map, filter, reduce). e. Create a program that manipulates objects and displays output. f. Implement a small JavaScript program combining functions, arrays, and objects.		
3.	React Introduction and JSX a. Create a React application using Vite/CRA and display "Welcome to MERN Stack Development". Display dynamic data using JSX. b. Declare variables and write a function to calculate the sum of two numbers and display the result. Apply basic CSS styling to a React component c. Create a simple functional component and render it in App.js. d. Create a React component that displays the current date and time dynamically using JavaScript		
4	React Components, Props, and State a. Create multiple components and render them on a single page. b. Pass data between components using props		

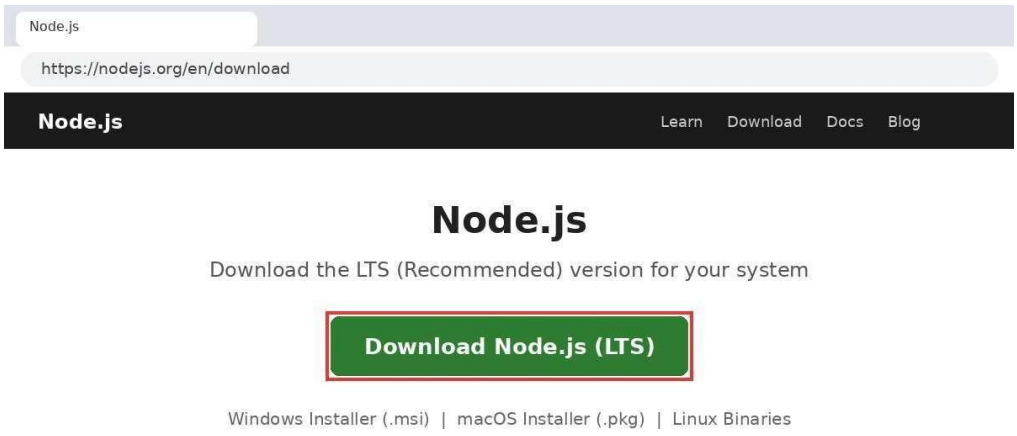
Practical No.	Topic	Date	Signature
	c. Use the useState hook to manage and update data. d. Create a form component to accept user input and display it dynamically. e. Display a list of items stored in the state using map(). f. Build a small interactive component (e.g., counter or to-do list).		
5.	React Routing and Data Handling a. Install and configure React Router in the React application. b. Create multiple pages and implement navigation between them. c. Use Link and Routes to move between pages. d. Fetch sample data from a public API using Axios. e. Display fetched data in table or list format. f. Implement search or filter functionality on displayed data.		
6.	Node.js and Express Fundamentals a. Install Express and create a simple server. b. Display a message on the browser using the server response. c. Create different routes in Express. d. Send a JSON response from the server. f. Run the server and observe the output in the console. e. Test routes using browser		
7.	MongoDB Basics a. Install and start MongoDB. b. Create a database and collection. c. Insert records in MongoDB. d. View records from the collection. e. Update and delete records. f. Connect MongoDB with Node.js.		
8.	API Creation and Integration a. Create a simple REST Ari using Express b. Implement a GET API to retrieve data. c. Fetch data from the API in a React application		

PRACTICAL 1

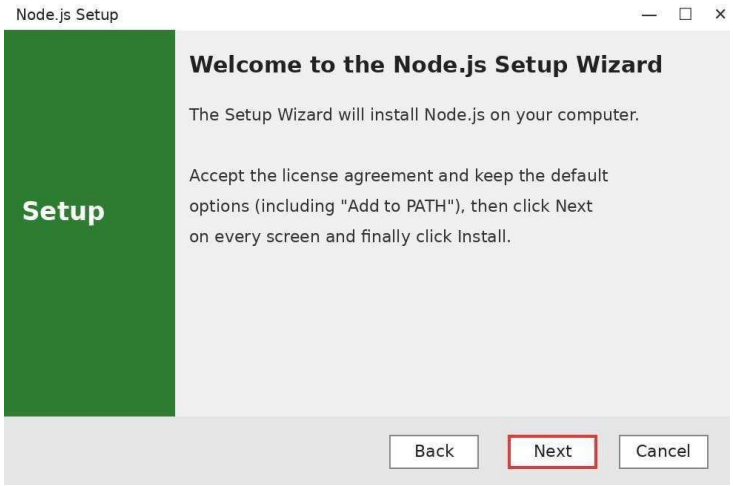
Aim:1(a) Install Node.js and Visual Studio Code, and verify installation.

Procedure :

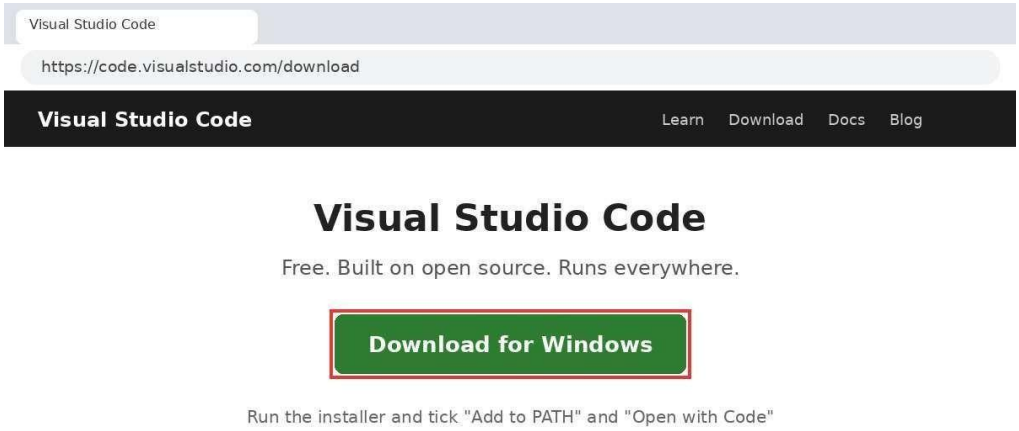
Step 1: Open https://nodejs.org in a browser and download the LTS (Recommended) installer for your operating system.



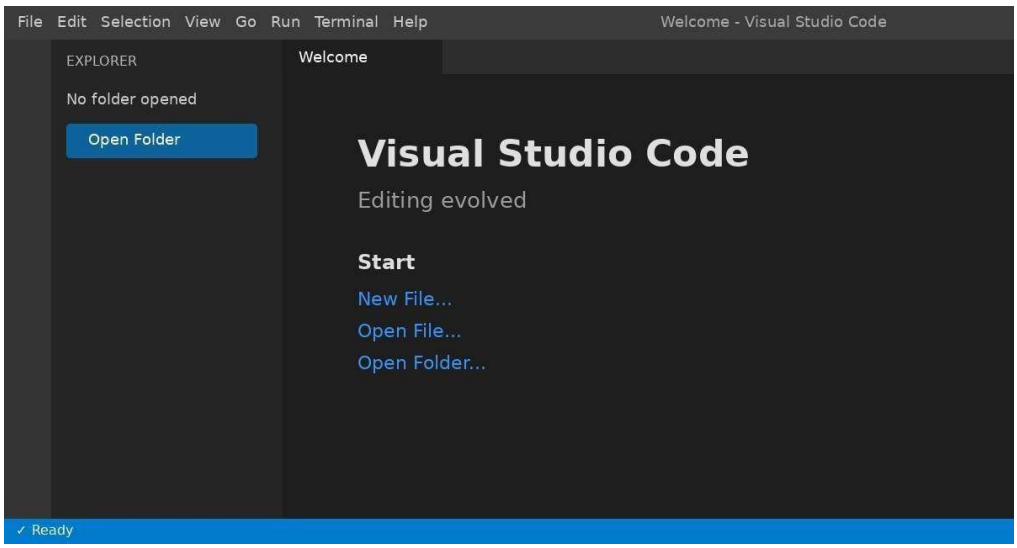
Step 2: Run the downloaded installer. Accept the license agreement, keep the default options (including "Add to PATH") and click Next on each screen, then click Install and Finish.



Step 3: Open https://code.visualstudio.com and download the installer for your system.



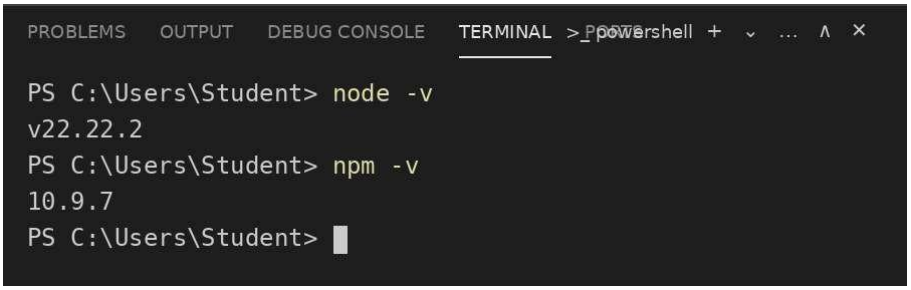
Step 4: Run the VS Code installer, tick “Add to PATH” and “Open with Code” and complete the installation. Launch Visual Studio Code to confirm that it opens.



Verification :

Step 5: Open a terminal (PowerShell / Command Prompt) and run node -v and npm -v. The installed version numbers are displayed, which confirms that Node.js and npm are installed correctly.

Output :



Aim : 1(b) Create a simple Node.js file and run it using the terminal.

Code : hello.js

Create a folder, open it in VS Code, create a new file named hello.js and type the code below.

FullStack > Practical1 > hello.js > ...

```
1  console.log("Hello, Full Stack Development!");
```

Output :

Open the terminal in the folder containing the file and run it with the node command.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS >_ powershell + ⌄ ... ∧ ×

```
PS D:\FullStack> cd D:\FullStack\Practical1
PS D:\FullStack\Practical1> dir
    Directory: D:\FullStack\Practical1

Mode   Name
----   ----
-a---  hello.js
PS D:\FullStack\Practical1> node hello.js
Hello, Full Stack Development!
PS D:\FullStack\Practical1>
```

Aim : 1(c) Write a Node.js script demonstrating basic JavaScript commands (console log, variables, and arithmetic operations)

Code : basic.js

FullStack > Practical1 > basic.js > ...

```
1  let number = 25;
2  if(number % 2===0){
3   console.log(number+' is Even');
4  }else{
5   console.log(number+' is Odd');
6  }
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS >_ powershell + ⌄ ... ∧ ×

```
PS D:\FullStack\Practical1> node basic.js
25 is Odd
PS D:\FullStack\Practical1>
```

Aim: 1(d) Create and run a simple program using variables and functions.

Code : func.js

FullStack > Practical1 > func.js > ...

```
1  function add(a,b)
2  {
3  return a+b;
4  }
5  console.log(add(15,25));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS >_ powershell + ... ^ ×

```
PS D:\FullStack\Practical1> node func.js
40
PS D:\FullStack\Practical1>
```

Aim : 1(e) Demonstrate use of operators and control statements in JavaScript.

Code : ops.js

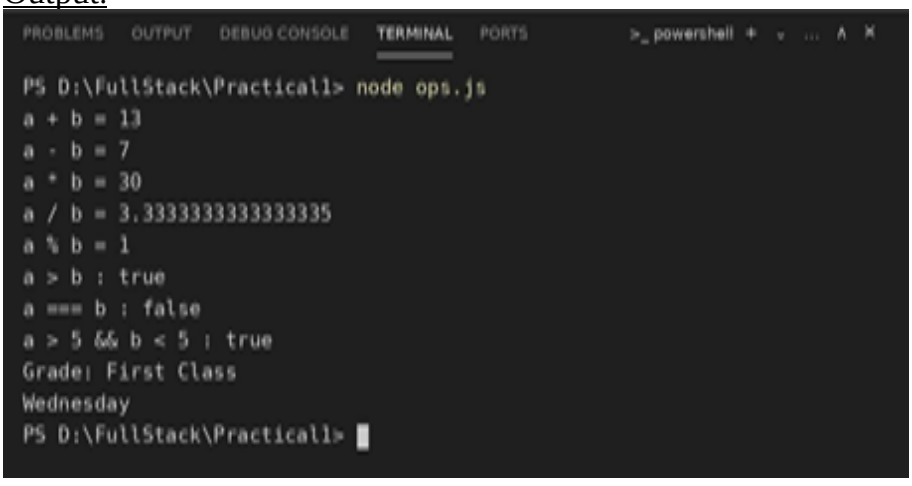
FullStack > Practical1 > ops.js > ...

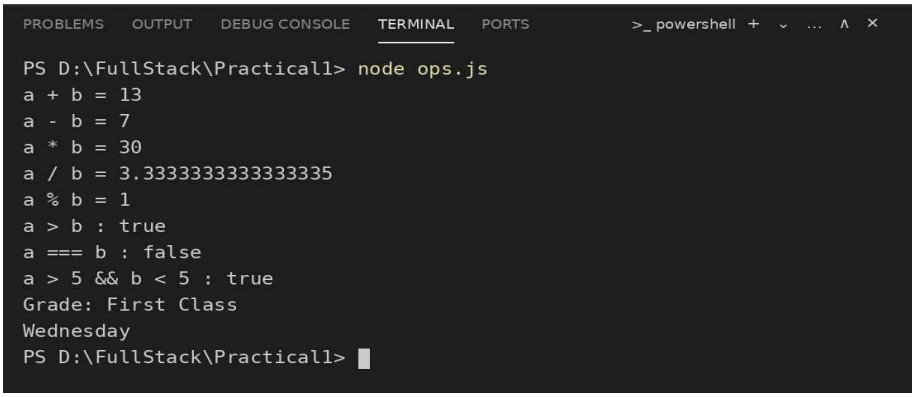
```
let a = 10, b = 3;
console.log("a + b =", a + b);
console.log("a - b =", a - b);
console.log("a * b =", a * b);
console.log("a / b =", a / b);
console.log("a % b =", a % b);
console.log("a > b :", a > b);
console.log("a === b :", a === b);
console.log("a > 5 && b < 5 :", a > 5 && b < 5);

let marks = 72;
if (marks >= 75) {
  console.log("Grade: Distinction");
} else if (marks >= 60) {
  console.log("Grade: First Class");
} else if (marks >= 40) {
  console.log("Grade: Pass");
} else {
  console.log("Grade: Fail");
}

let day = 3;
switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  case 3: console.log("Wednesday"); break;
  default: console.log("Invalid day");
}
```

Output:





Aim: 1(f) Write programs using loops and template literals.

Code: loop.js

FullStack > Practical1 > loop.js > ...

```
let s={name:'Rahul',age:20};
for(let k in s)
console.log(k,s[k]);
```

Output:



Code: template.js

FullStack > Practical1 > template.js > ...

```
let name = 'Rahul';
let age = 20;
console.log(`My name is ${name} and I am ${age} years old.`);
for (let i = 1; i <= 5; i++) {
  console.log(`2 x ${i} = ${2 * i}`);
}
```

Output:



PRACTICAL 2

Aim : 2(a) Create functions and call them with parameters.

Code : square.js

FullStack > Practical2 > square.js > ...

```
1  const square=n=>n*n;
2   console.log(square(8));
```

Output :



Aim : 2(b) Work with arrays and objects in JavaScript.

Code : array.js (Array)

FullStack > Practical2 > array.js > ...

```
1  let arr=[10,20,30];
2  arr.push(40);
3  arr.pop();
4  console.log(arr);
```

Output :



Code: object.js (Object)

FullStack > Practical2 > object.js > ...

```
let student={name:'Rahul',age:20,course:'Node.js'};
 console.log(student);
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        >_ powershell + ⌄ ... ∧ ×

PS D:\FullStack\Practical2> node object.js
{ name: 'Rahul', age: 20, course: 'Node.js' }
PS D:\FullStack\Practical2>
```

Aim: 2(d) Write a program using array methods (map, filter, reduce).

Code: methods.js

FullStack > Practical2 > methods.js > ...

```
let numbers = [1, 2, 3, 4, 5, 6];
let doubled = numbers.map(n => n * 2);
let evens = numbers.filter(n => n % 2 === 0);
let sum = numbers.reduce((total, n) => total + n, 0);
console.log("Original:", numbers);
console.log("map    :", doubled);
console.log("filter :", evens);
console.log("reduce :", sum);
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        >_ powershell + ⌄ ... ∧ ×

PS D:\FullStack\Practical2> node methods.js
Original: [ 1, 2, 3, 4, 5, 6 ]
map    : [ 2, 4, 6, 8, 10, 12 ]
filter : [ 2, 4, 6 ]
reduce : 21
PS D:\FullStack\Practical2>
```

Aim: 2(e) Create a program that manipulates objects and displays output.

Code: objmanip.js

FullStack > Practical2 > objmanip.js > ...

```
let student = {name: 'Rahul', age: 20, course: 'Node.js'};
student.age = 21;
student.city = 'Mumbai';
delete student.course;
console.log(student);
console.log(Object.keys(student));
console.log(Object.values(student));
console.log('name' in student);
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS >_ powershell + ⌄ ... ∧ ×

```
PS D:\FullStack\Practical2> node objmanip.js
{ name: 'Rahul', age: 21, city: 'Mumbai' }
[ 'name', 'age', 'city' ]
[ 'Rahul', 21, 'Mumbai' ]
true
PS D:\FullStack\Practical2>
```

Aim: 2(f) Implement a small JavaScript program combining functions, arrays, and objects.

Code : combined.js

FullStack > Practical2 > combined.js > ...

```
const students = [
  {name: 'Rahul', marks: 78},
  {name: 'Anita', marks: 92},
  {name: 'Karan', marks: 55}
];
function getGrade(marks) {
  if (marks >= 75) return 'A';
  else if (marks >= 60) return 'B';
  return 'C';
}
students.forEach(s => {
  console.log(`${s.name}: ${s.marks} (Grade ${getGrade(s.marks)})`);
});
const total = students.reduce((t, s) => t + s.marks, 0);
console.log('Average marks:', (total / students.length).toFixed(2));
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS >_ powershell + ⌄ ... ∧ ×

```
PS D:\FullStack\Practical2> node combined.js
Rahul: 78 (Grade A)
Anita: 92 (Grade A)
Karan: 55 (Grade C)
Average marks: 75.00
PS D:\FullStack\Practical2>
```

PRACTICAL 3

Aim : 3(a) . Create a React application using Vite/CRA and display ”Welcome to MERN Stack Development“

Code : App.jsx

```jsx
function App() {
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
    </div>
  );
}

export default App;
```

: Output

Welcome to MERN Stack Development

Aim: 3 (b). Display dynamic data using JSX

Code : App.jsx

App.jsx react-pract react-practical

My_Fullstack_Program_1 > src > App.jsx > ...

```jsx
import { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [course, setCourse] = useState("");

  return (
    <div>
      <h1>Student Information</h1>

      <label>Enter Name: </label>

      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <br />
      <br />
```

:Continue

```jsx
      <label>Enter Course: </label>

      <input
        type="text"
        value={course}
        onChange={(e) => setCourse(e.target.value)}
      />

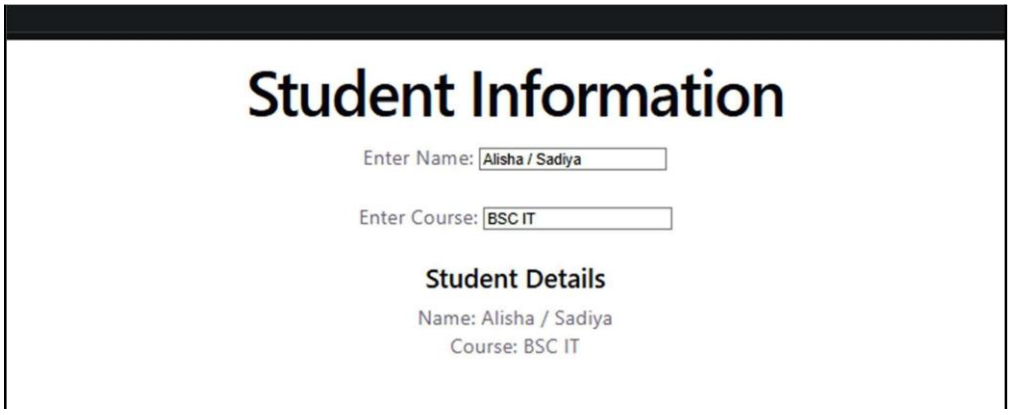
      <br />
      <br />

      <h2>Student Details</h2>

      <p>Name: {name}</p>
      <p>Course: {course}</p>
    </div>
  );
}

export default App;
```

: Output



Aim : 3 (c) Declare variables and write a function to calculate the sum of .two numbers and display the result

Code : App.jsx

```jsx
import { useState } from "react";

function App() {
  const [num1, setNum1] = useState("");
  const [num2, setNum2] = useState("");
  const [result, setResult] = useState("");

  function calculateSum() {
    const number1 = Number(num1);
    const number2 = Number(num2);

    const sum = number1 + number2;

    setResult(sum);
  }

  return (
    <div>
      <h1>Addition of Two Numbers</h1>

      <label>Enter First Number: </label>

      <input
        type="number"
        value={num1}
        onChange={(e) => setNum1(e.target.value)}
      />

      <br />
      <br />

      <label>Enter Second Number: </label>

      <input
        type="number"
        value={num2}
        onChange={(e) => setNum2(e.target.value)}
      />

      <br />
      <br />
```

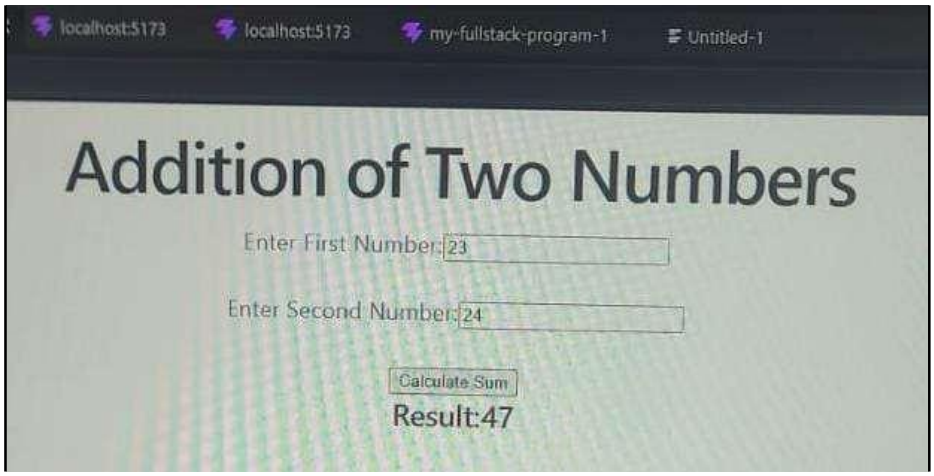
:Continue

```
        <button onClick={calculateSum}>
          Calculate Sum
        </button>

        <h2>Result: {result}</h2>
      </div>
    );
  }

export default App;
```

:OUTPUT



Aim : 3 (d) Apply basic CSS styling to a React component.

Code : App.jsx

```jsx
import { useState } from "react";
import "./App.css";

function App() {
  const [name, setName] = useState("");

  return (
    <div className="container">
      <h1>React CSS Example</h1>

      <label>Enter Your Name: </label>

      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <h2>Hello {name}</h2>
    </div>
  );
}

export default App;
```

App.css :

```css
.container {
  width: 500px;
  margin: 50px auto;
  padding: 30px;
  text-align: center;
  border: 2px solid blueviolet;
  border-radius: 10px;
  font-family: Arial;
}

h1 {
  font-size: 30px;
}

input {
  padding: 10px;
  font-size: 16px;
}

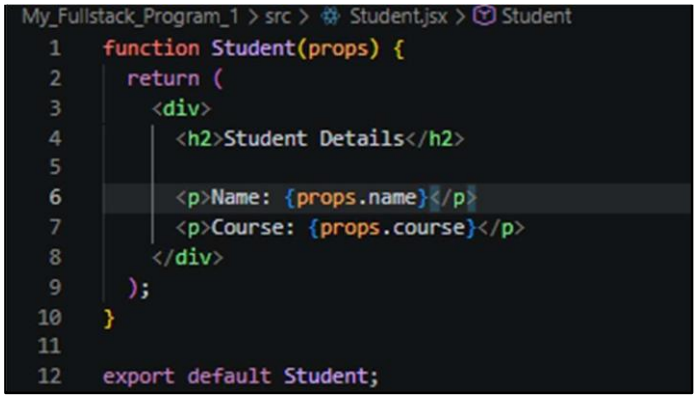
h2 {
  margin-top: 20px;
}
```

: Output



Aim: 3(e) Create a simple functional component and render it in . App.js

Code : Student.jsx



: App.jsx



:Continue

```
      <label>Enter Course: </label>

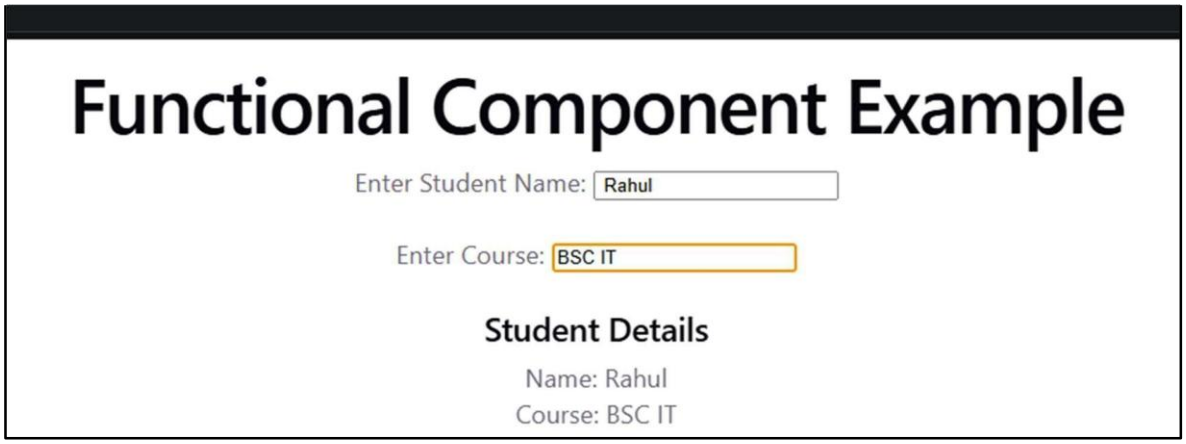
      <input
        type="text"
        value={course}
        onChange={(e) => setCourse(e.target.value)}
      />

      <br />
      <br />

      <Student
        name={name}
        course={course}
      />
    </div>
  );
}

export default App;
```

: Output



Aim : 3(f) Create a React component that displays the current date and . time dynamically using JavaScript

Code : App.jsx

```jsx
import { useState, useEffect } from "react";

function App() {
  const [dateTime, setDateTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => {
      setDateTime(new Date());
    }, 1000);

    return () => clearInterval(timer);
  }, []);

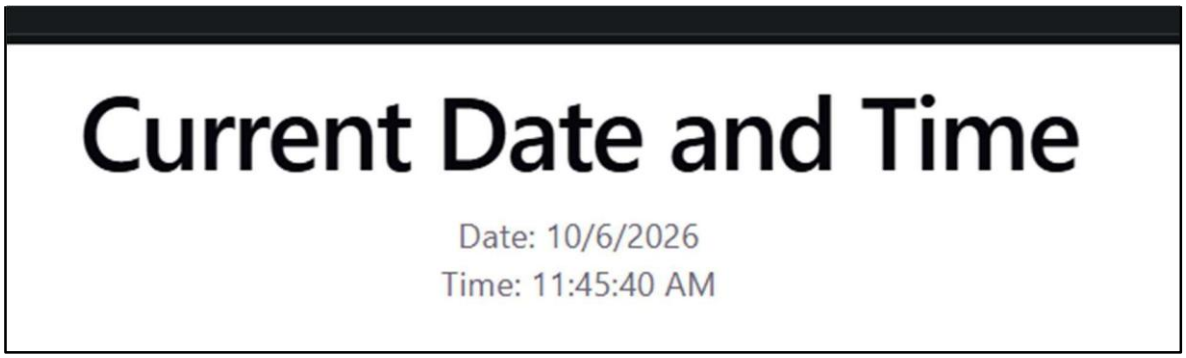
  return (
    <div>
      <h1>Current Date and Time</h1>

      <p>
        Date: {dateTime.toLocaleDateString()}
      </p>

      <p>
        Time: {dateTime.toLocaleTimeString()}
      </p>
    </div>
  );
}

export default App;
```

: Output



PRACTICAL 4

Aim: 4(a) Create Multiple Components and Render Them on a Single. Page

Code: App.jsx

My_Fullstack_Program_1 > src > App.jsx > App

```jsx
import Header from "./Header";
import Student from "./Student";
import Footer from "./Footer";

function App() {
  return (
    <div>
      <Header />
      <Student />
      <Footer />
    </div>
  );
}
```

: Header.jsx

My_Fullstack_Program_1 > src > Header.jsx > ...

```jsx
function Header() {
  return (
    <header>
      <h1>TYIT React Practical</h1>
    </header>
  );
}

export default Header;
```

Student.jsx

My_Fullstack_Program_1 > src > Student.jsx > Student

```jsx
function Student() {
  return (
    <div>
      <h2>Student Information</h2>
      <p>Name: Rahul</p>
      <p>Course: TYBSc IT</p>
    </div>
  );
}

export default Student;
```

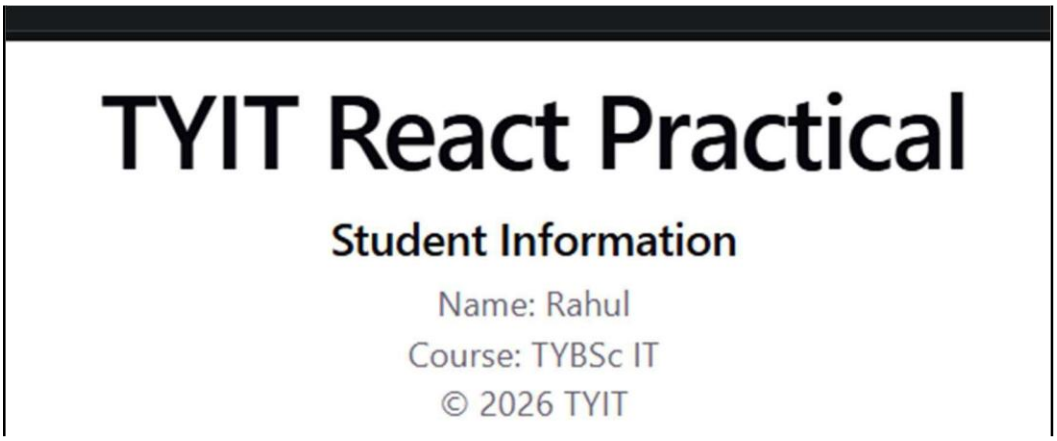
: Footer.jsx

My_Fullstack_Program_1 > src > Footer.jsx > ...

```jsx
function Footer() {
  return (
    <footer>
      <p>© 2026 TYIT</p>
    </footer>
  );
}

export default Footer;
```

: Output



Aim : 4 (b) Pass Data Between Components Using Props

Code : App.jsx

React-pract > src > App.jsx > App

```jsx
import Student from "./Student";

function App() {
  return (
    <div>
      <h1>Student Details</h1>

      <Student
        name="Rahul"
        course="BSc IT"
        marks="85"
      />
    </div>
  );
}
export default App;
```

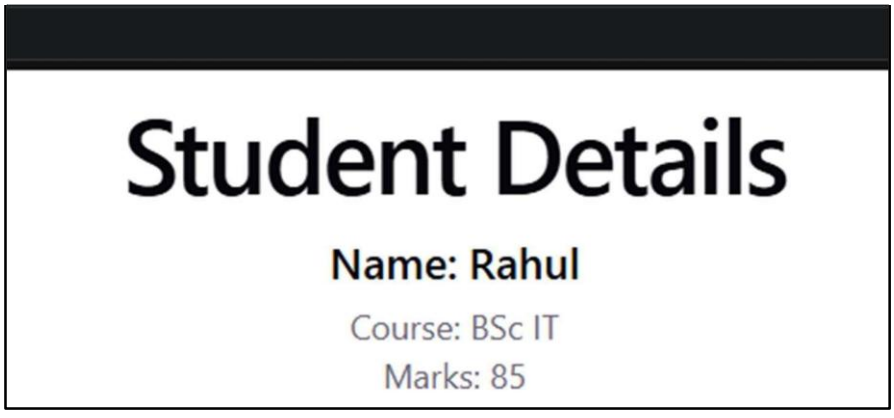
: Student.jsx

React-pract > src > Student.jsx > ...

```jsx
function Student(props) {
  return (
    <div>
      <h2>Name: {props.name}</h2>
      <p>Course: {props.course}</p>
      <p>Marks: {props.marks}</p>
    </div>
  );
}

export default Student;
```

: Output



Aim: 4 (c) Use the useState Hook to Manage and Update Data

Code : App.jsx

```jsx
import { useState } from "react";

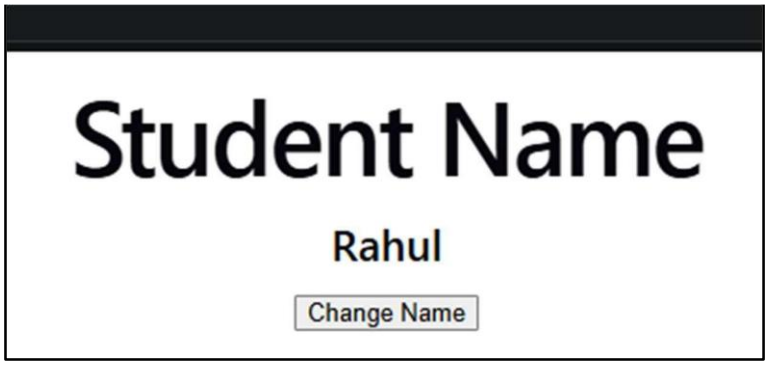
function App() {
  const [name, setName] = useState("Rahul");

  return (
    <div>
      <h1>Student Name</h1>

      <h2>{name}</h2>

      <button onClick={() => setName("Amit")}>
        Change Name
      </button>
    </div>
  );
}
export default App;
```

: Output



:After clicking the button



Aim: 4 (d) Create a Form Component to Accept User Input and Display It . Dynamically

Code : App.jsx

React-pract > src > App.jsx > App

```jsx
import { useState } from "react";

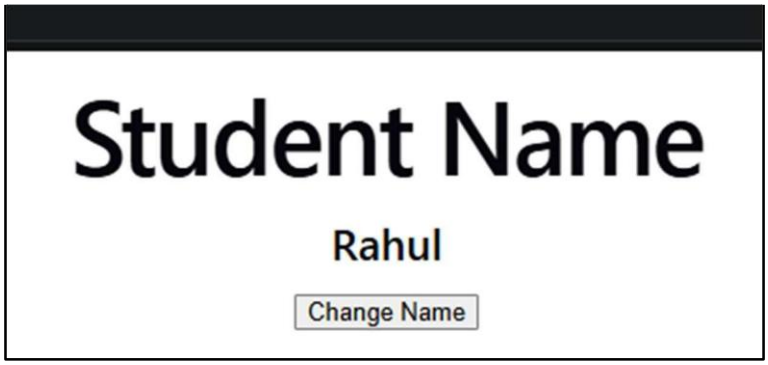
function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <h1>Student Form</h1>

      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <h2>Hello, {name}</h2>
    </div>
  );
}
export default App;
```

: Output



Aim: 4 (e) Display a List of Items Stored in State Using map

Code : App.jsx

React-pract > src > App.jsx > ...

```jsx
import { useState } from "react";

function App() {
  const [students] = useState([
    "Rahul",
    "Amit",
    "Priya",
    "Sneha"
  ]);
  return (
    <div>
      <h1>Student List</h1>

      <ul>
        {students.map((student, index) => (
          <li key={index}>{student}</li>
        ))}
      </ul>
    </div>
  );
}
export default App;
```

Output:



Aim : 4 (f) Build a Small Interactive Component – Counter

Code : App.jsx

```
React-pract > src > App.jsx > ...
function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Counter Application</h1>

      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

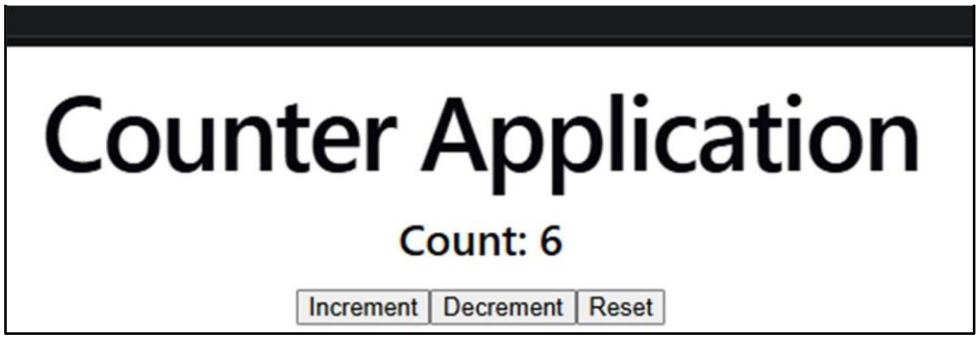
      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}
export default App;
```

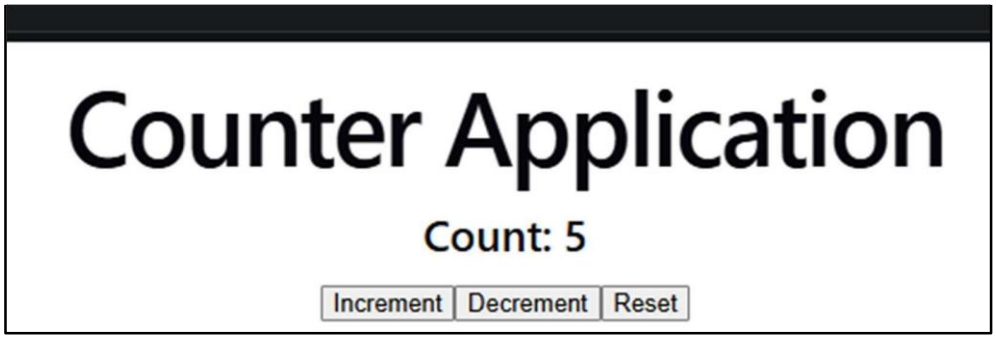
Output:



Increment:



Decrement:



PRACTICAL 5

Aim : 5(a) Install and Configure React Router

Code: App.jsx

routing-app > App.jsx > ...

```jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "../React-pract/src/Home";
import About from "../React-pract/src/About";
import Contact from "../React-pract/src/Contact";
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Home.jsx:

routing-app > src > Home.jsx > Home

```jsx
function Home() {
  return <h1>Welcome to Home Page</h1>;
}
export default Home;

```

About.jsx:

routing-app > src > About.jsx > ...

```jsx
function About() {
  return <h1>About Us</h1>;
}
export default About;

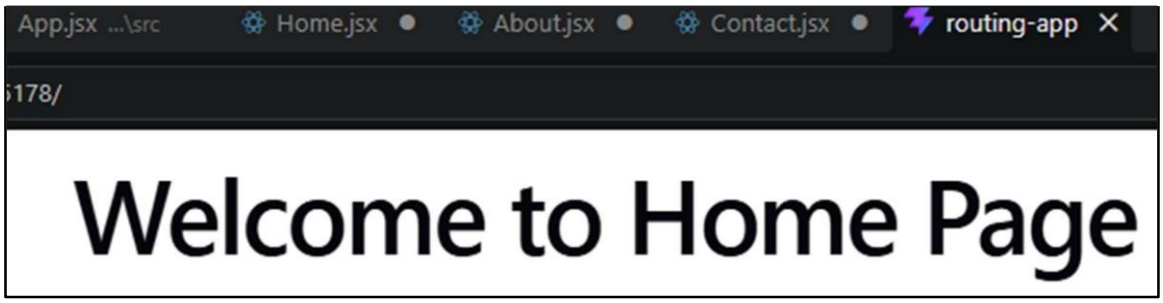
```

Contact.jsx:

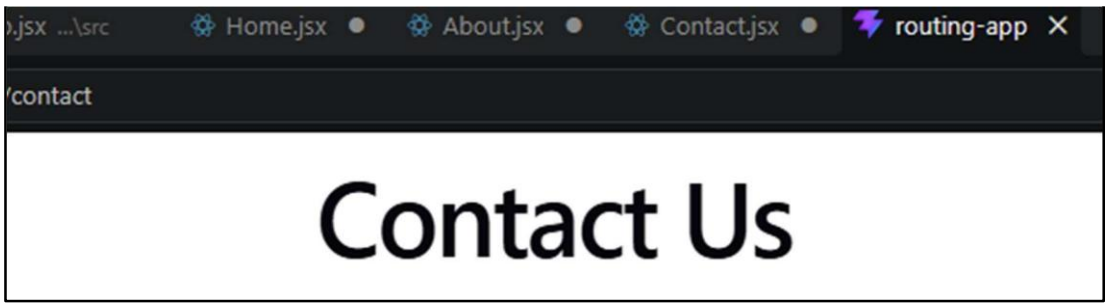
routing-app > src > Contact.jsx > ...

```jsx
function Contact() {
  return <h1>Contact Us</h1>;
}
export default Contact;
```

Output:







Aim : 5(b) Create Multiple Pages and Implement Navigation

Code : App.jsx

App.jsx | Home.jsx | About.jsx | Contact.jsx | routing-app

routing-app > App.jsx > ...

```jsx
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Home.jsx:

App.jsx | Home.jsx | About.jsx

routing-app > src > Home.jsx > ...

```jsx
function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <p>Welcome to our website.</p>
    </div>
  );
}
export default Home;
```

About.jsx:

routing-app > src > About.jsx > ...

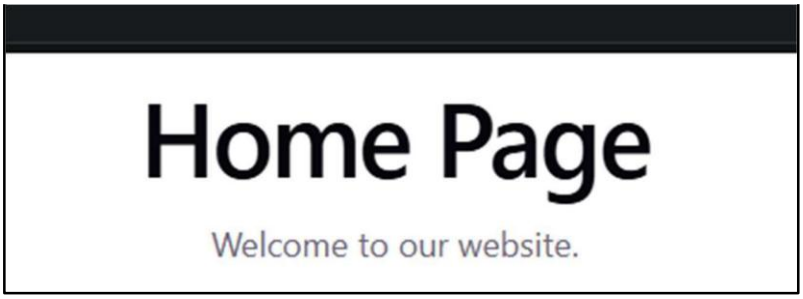
```jsx
function About() {
  return (
    <div>
      <h1>About Page</h1>
      <p>This is the About page.</p>
    </div>
  );
}
export default About;
```

Contact.jsx:

routing-app > src > Contact.jsx > ...

```jsx
function Contact() {
  return (
    <div>
      <h1>Contact Page</h1>
      <p>This is the Contact page.</p>
    </div>
  );
}
export default Contact;
```

Output:







The URLs will be:
/ http://localhost:5173

http://localhost:5173/about

http://localhost:5173/contact

Aim: 5(c) Use Link and Routes to Move Between Pages

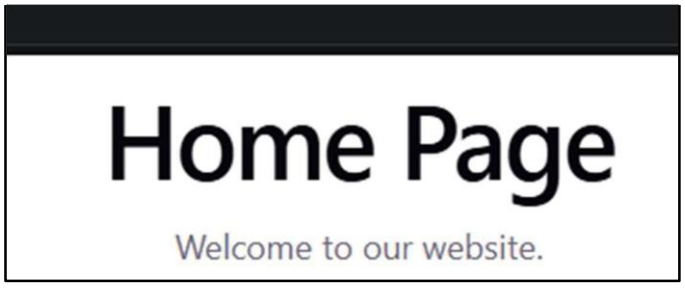
Code : App.jsx

routing-app > App.jsx > ...

```jsx
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";
function Home() {
  return <h2>Welcome to Home Page</h2>;
}
function About() {
  return <h2>Welcome to About Page</h2>;
}
function Contact() {
  return <h2>Welcome to Contact Page</h2>;
}
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> {" | "}
        <Link to="/about">About</Link> {" | "}
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Output:

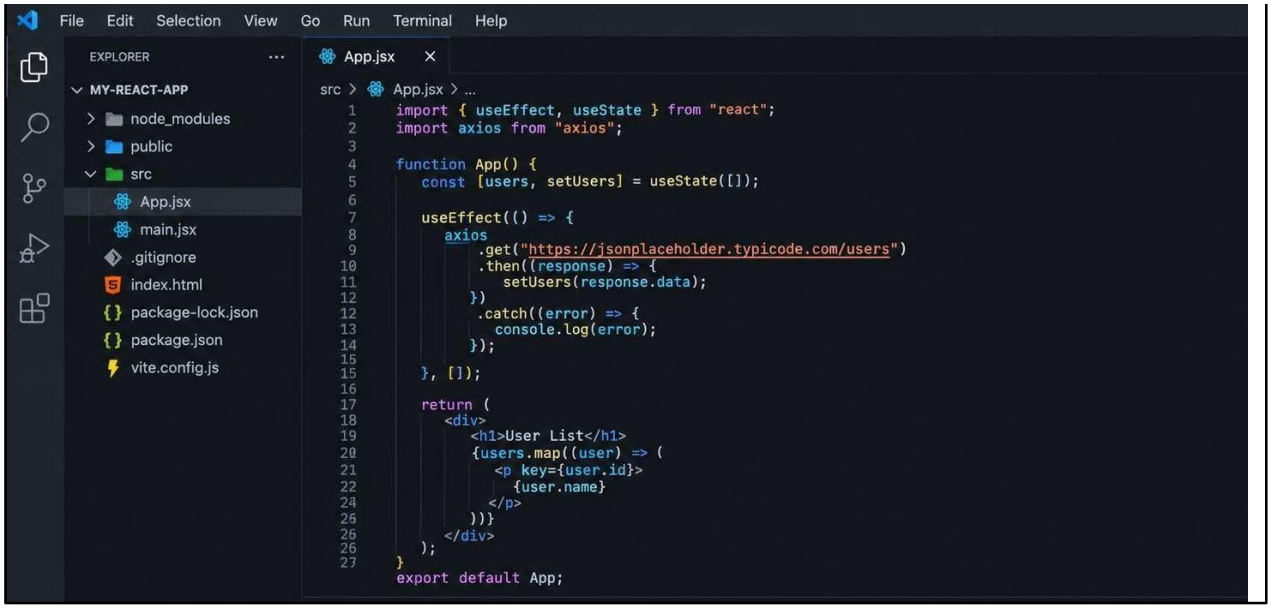



About Page
This is the About page.


Contact Page
This is the Contact page.

Aim : 5(d) Fetch Sample Data from a Public API Using Axios

Code : App.jsx

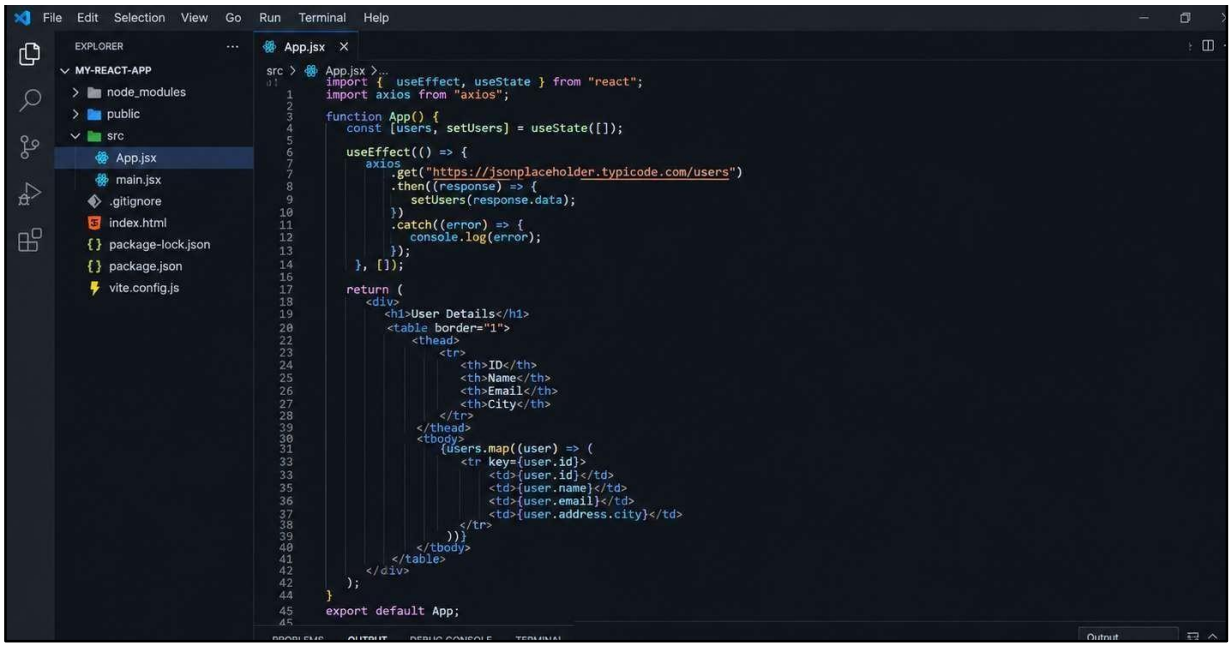


Output:



Aim : 5(e) Display Fetched Data in Table Format

Code : App.jsx

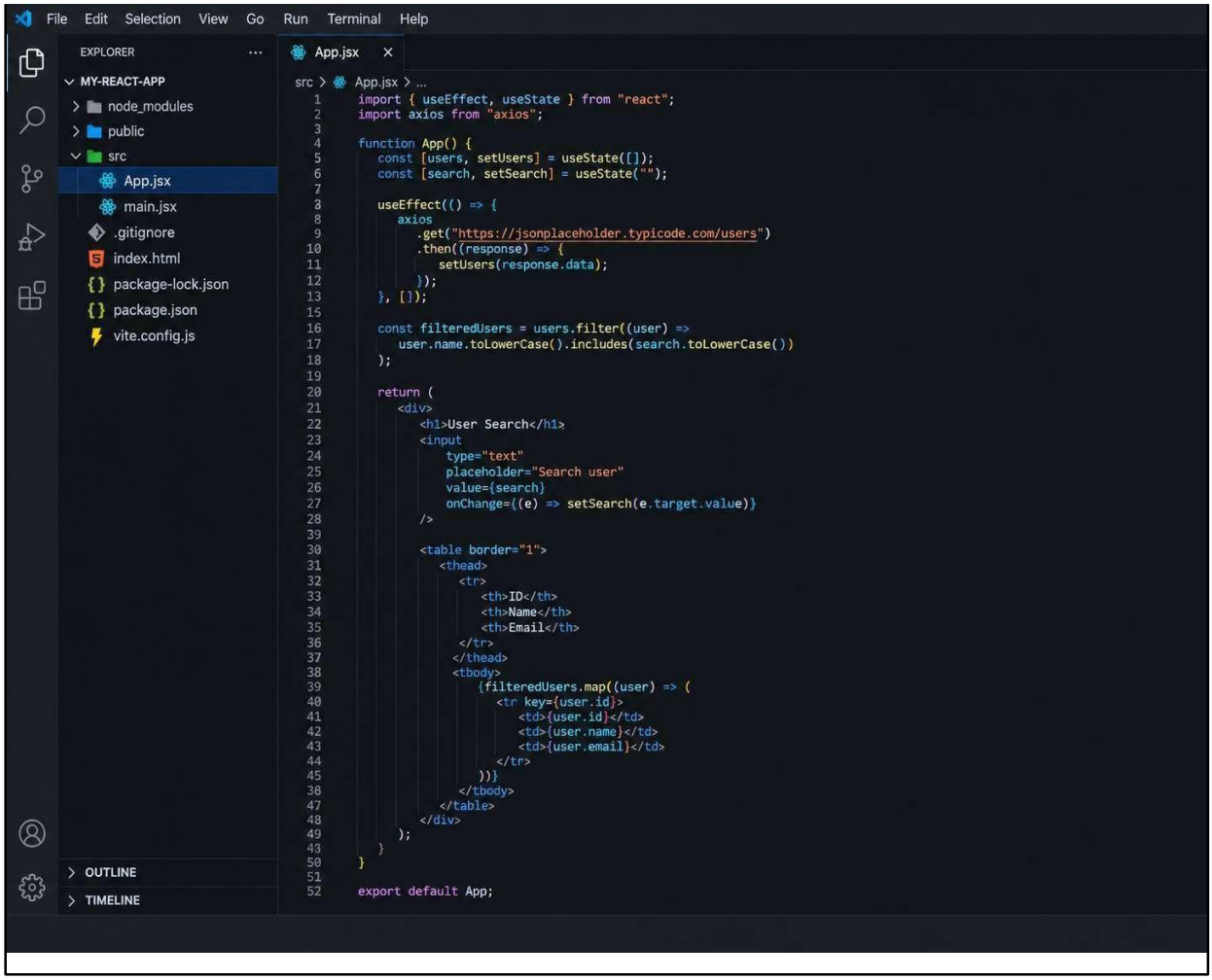


Output:

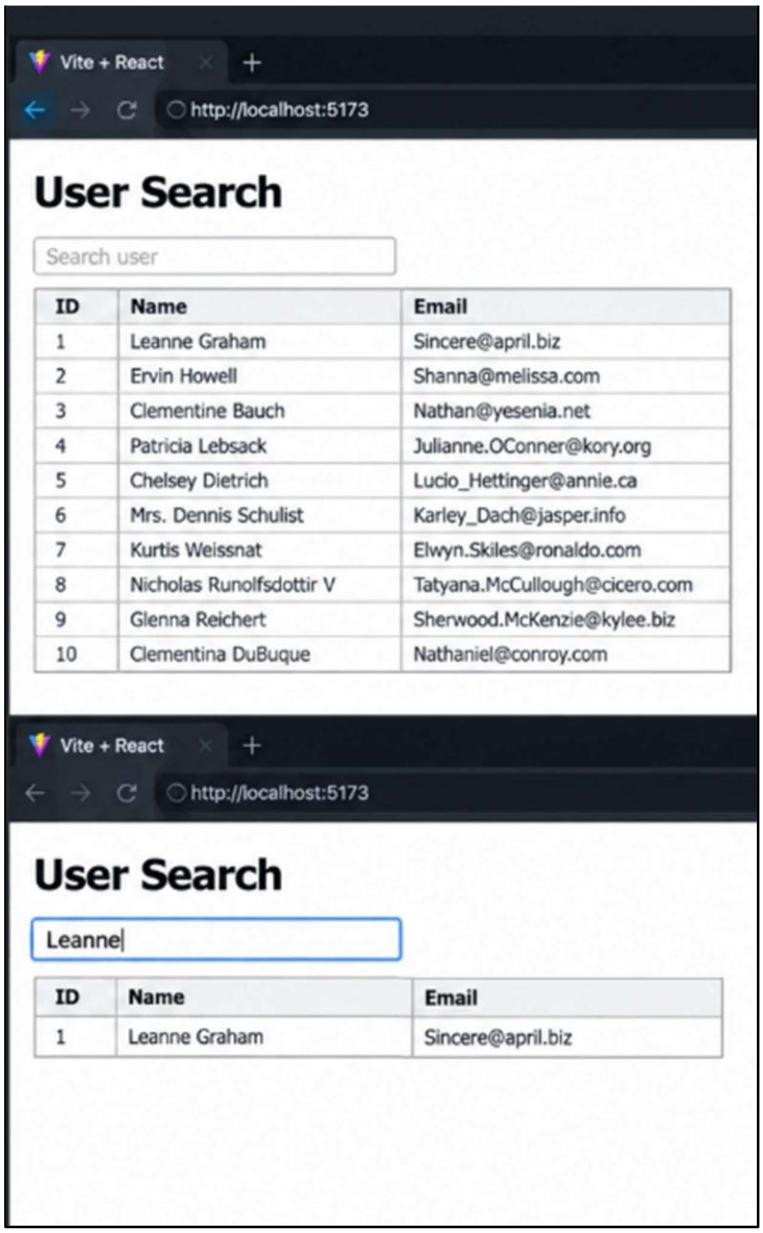


. Aim : 5(f) Implement Search or Filter Functionality

Code : App.jsx



Output::


Vite + React
http://localhost:5173
User Search
Search user
ID Name Email
1 Leanne Graham Sincere@april.biz
2 Ervin Howell Shanna@melissa.com
3 Clementine Bauch Nathan@yesenia.net
4 Patricia Lebsack Julianne.OConner@kory.org
5 Chelsey Dietrich Lucio_Hettinger@annie.ca
6 Mrs. Dennis Schulist Karley_Dach@jasper.info
7 Kurtis Weissnat Elwyn.Skiles@ronaldo.com
8 Nicholas Runolfsdottir V Tatyana.McCullough@cicero.com
9 Glenna Reichert Sherwood.McKenzie@kylee.biz
10 Clementina DuBuque Nathaniel@conroy.com
Vite + React
http://localhost:5173
User Search
Leanne
ID Name Email
1 Leanne Graham Sincere@april.biz

PRACTICAL 6

Aim:6 To create a basic server using Node.js and Express , Create Routes , return Json response and test them using a browser/Postman

Code:Server.js

server.js

backend > server.js > express

```
const express = require("express");

const app = express();

const PORT = 5000;

// Middleware
app.use(express.json());

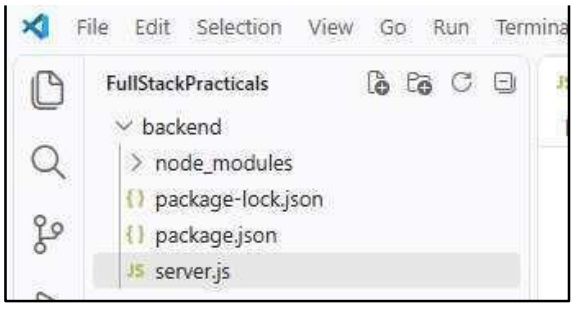
// Home route
app.get("/", (req, res) => {
    res.send("Welcome to Node.js and Express");
});

// About route
app.get("/about", (req, res) => {
    res.send("This is the About page");
});

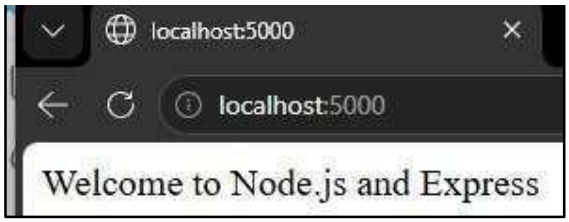
// JSON route
app.get("/api/test", (req, res) => {
    res.json({
        message: "Express API is working successfully"
    });
});

// Start server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});

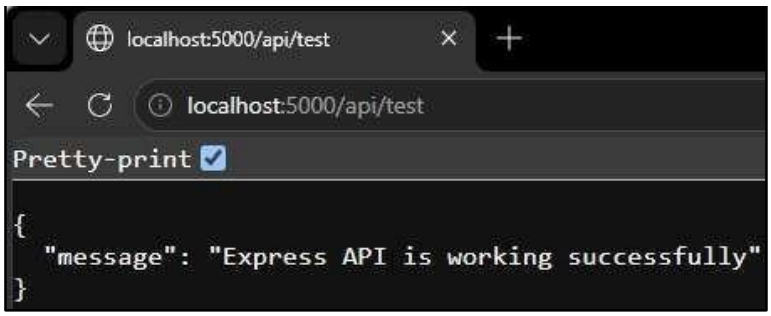
```



Output:







You have demonstrated:

- Express installation
- Server creation
- Browser response
- Multiple routes ● JSON response
- Server console output

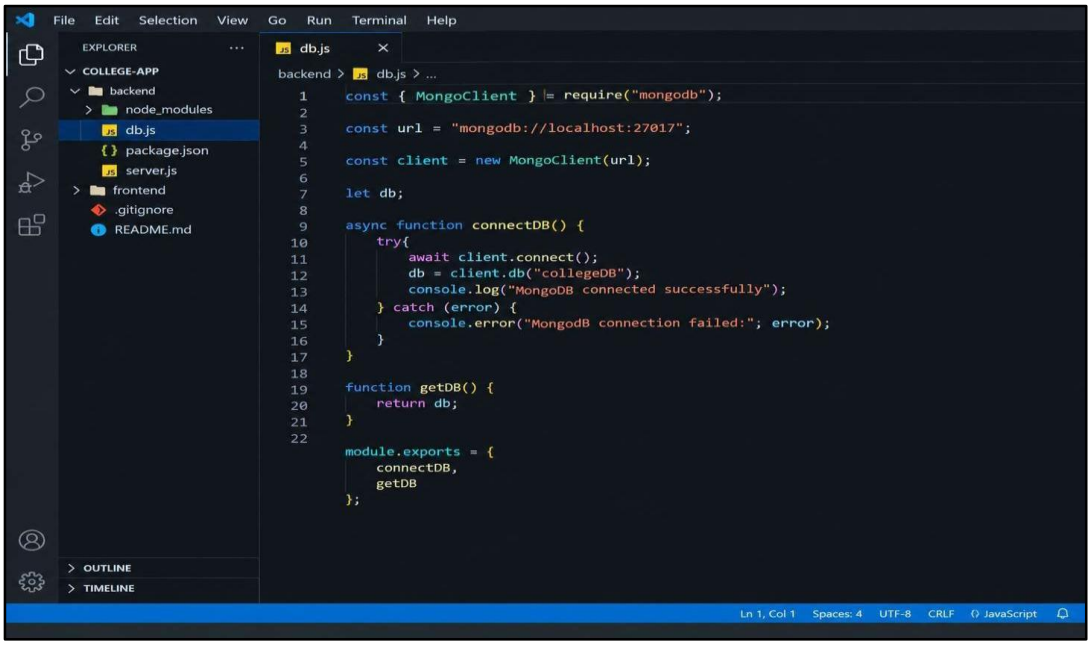
PRACTICAL 7

PRACTICAL 7 — MongoDB Basics

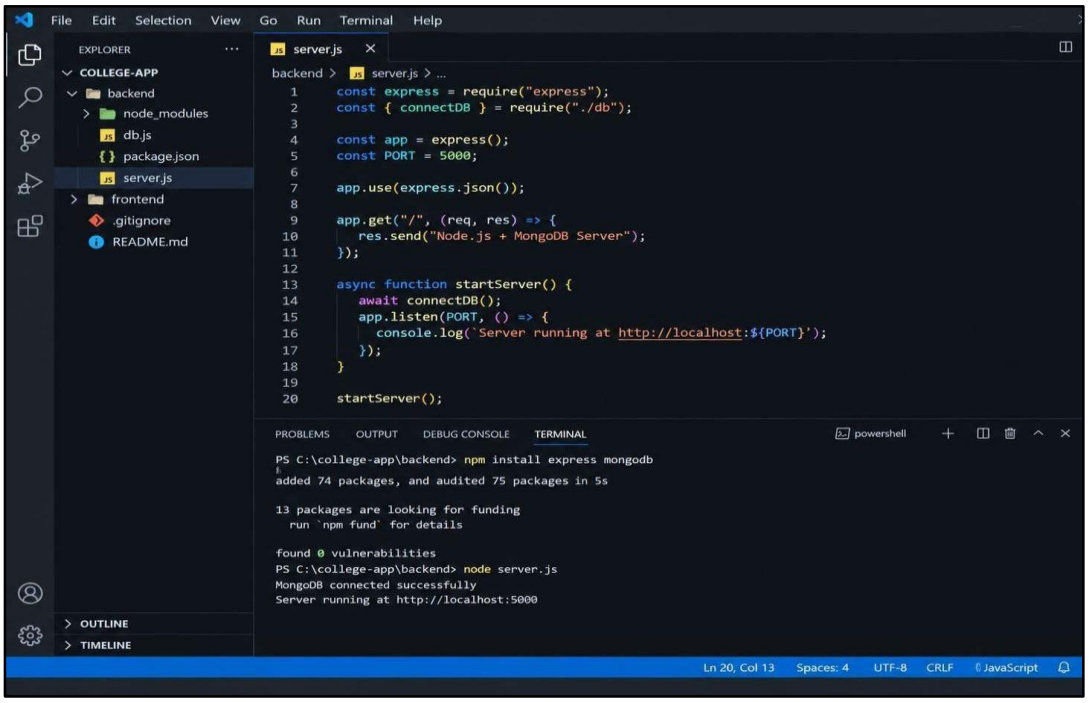
Aim: To create a MongoDB database and collection, . insert/view/update/delete records and connect MongoDB with Node.js

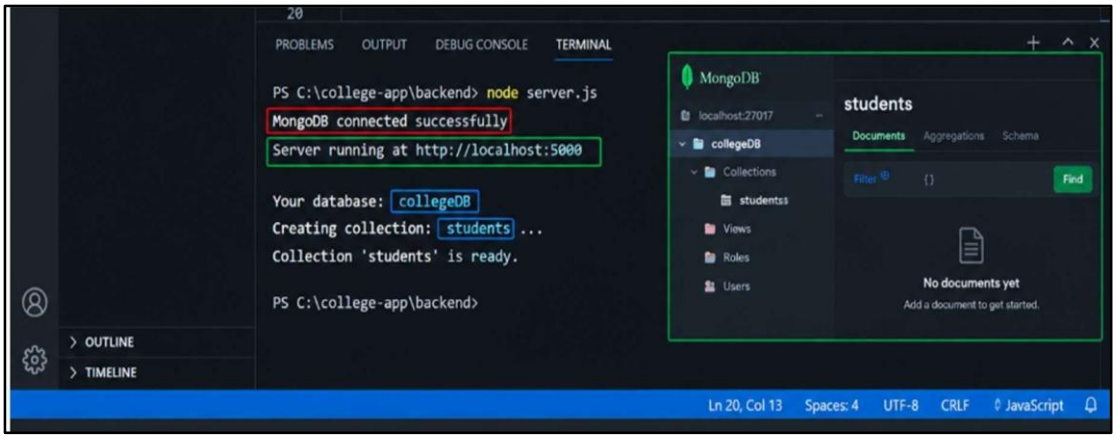
Code:

Create db.js



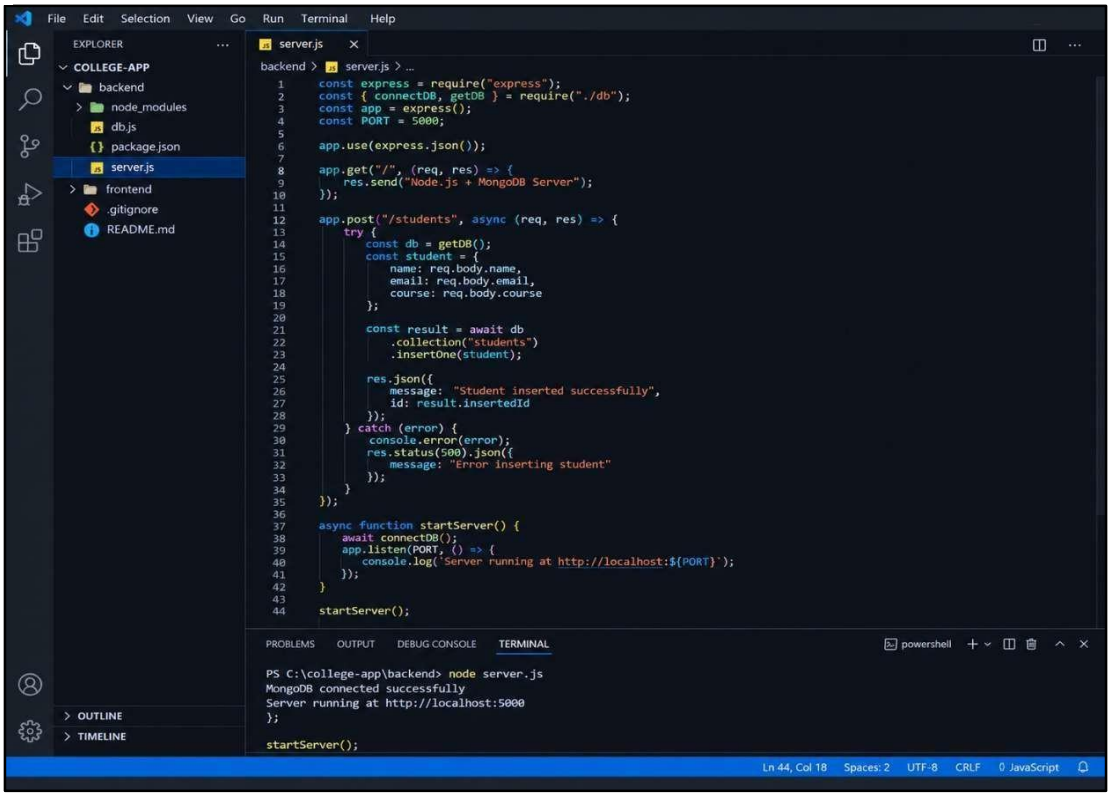
Modify : server.js



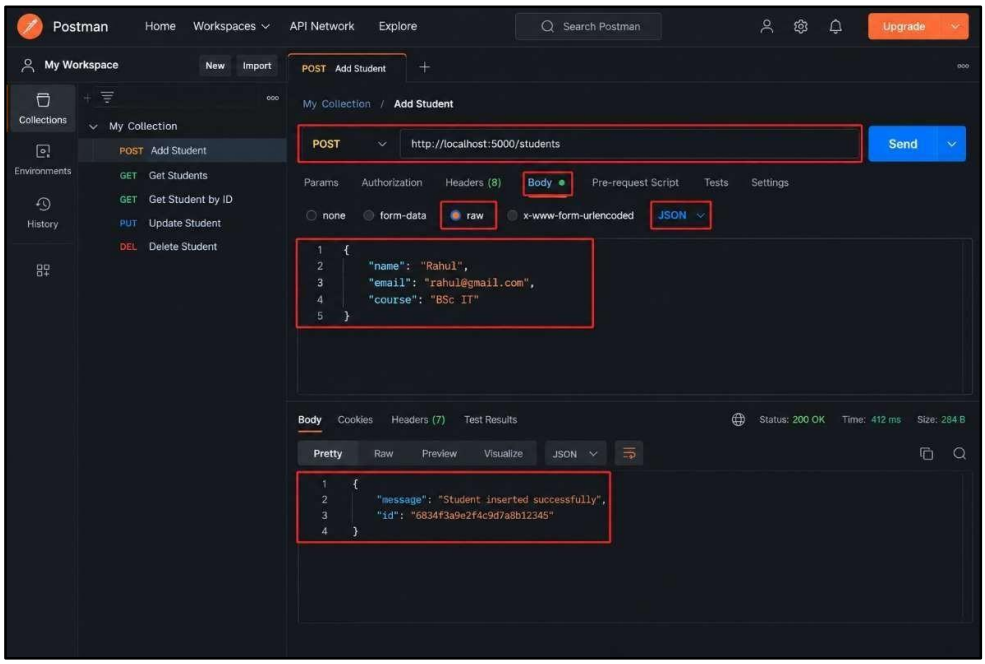


Step 7 — Insert Data

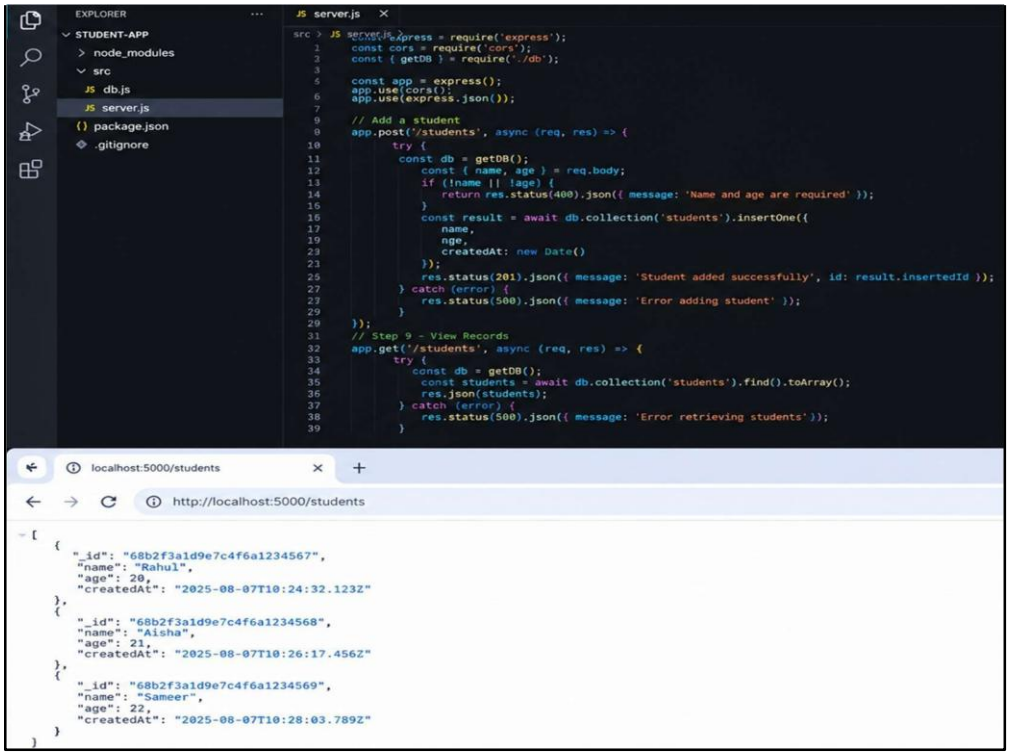
Modify server.js:



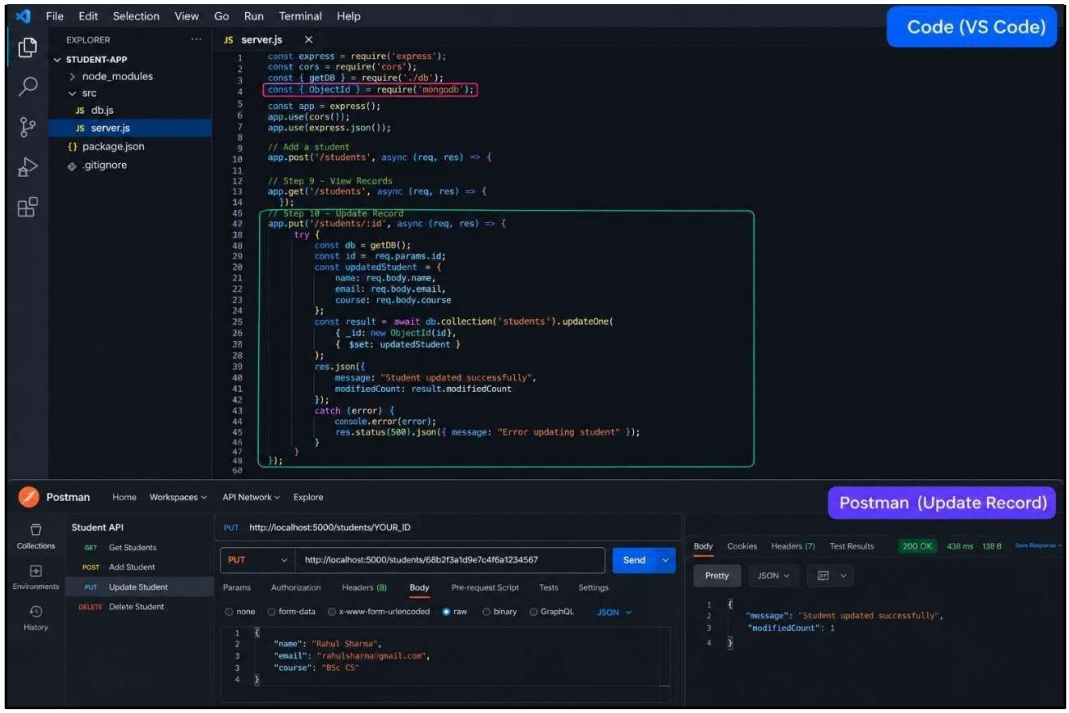
Step 8 — Test POST in Postman



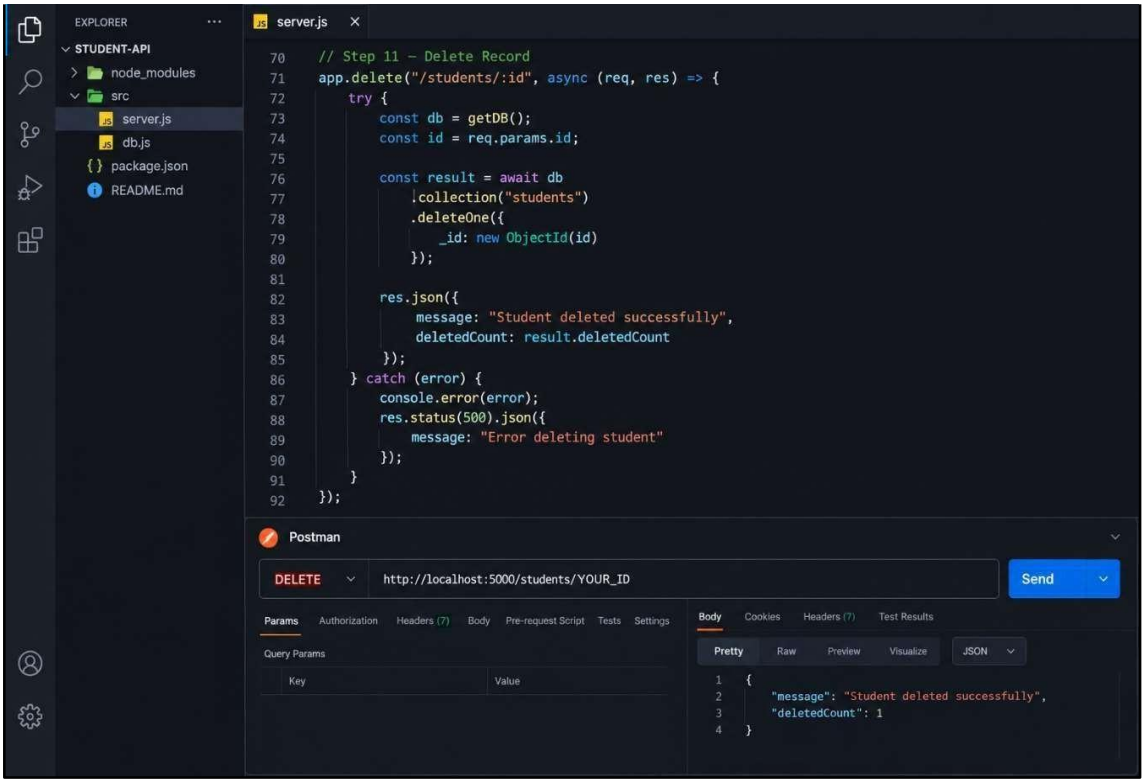
Step 9 — View Records



Step 10 — Update Record



Step 11 — Delete Record



You have demonstrated:

- MongoDB
- ↓
- Database
- ↓
- Collection
- ↓
- Insert
- ↓
- View
- ↓
- Update
- ↓
- Delete
- ↓
- Node.js connection

PRACTICAL 8PRACTICAL 8 — API Creation and Integration

Aim: To create a REST API using Express and integrate it with a React .application

Now we'll create the React frontend

Step 1 — Go to Main Folder

.Open a new terminal

: You should be in

FullStackPracticals/backend

:Go back

.. cd

:Now you are in

FullStackPracticals

Step 2 — Create React Application

:Run npm create vite@latest frontend ---- template

react

:Then cd

frontend

: Install dependencies npm

install

Vite provides the standard React project scaffolding used here. Vite Guide

Step 3 — Start React npm

run dev

:You'll see something like

/Local: http://localhost:5173

:Open

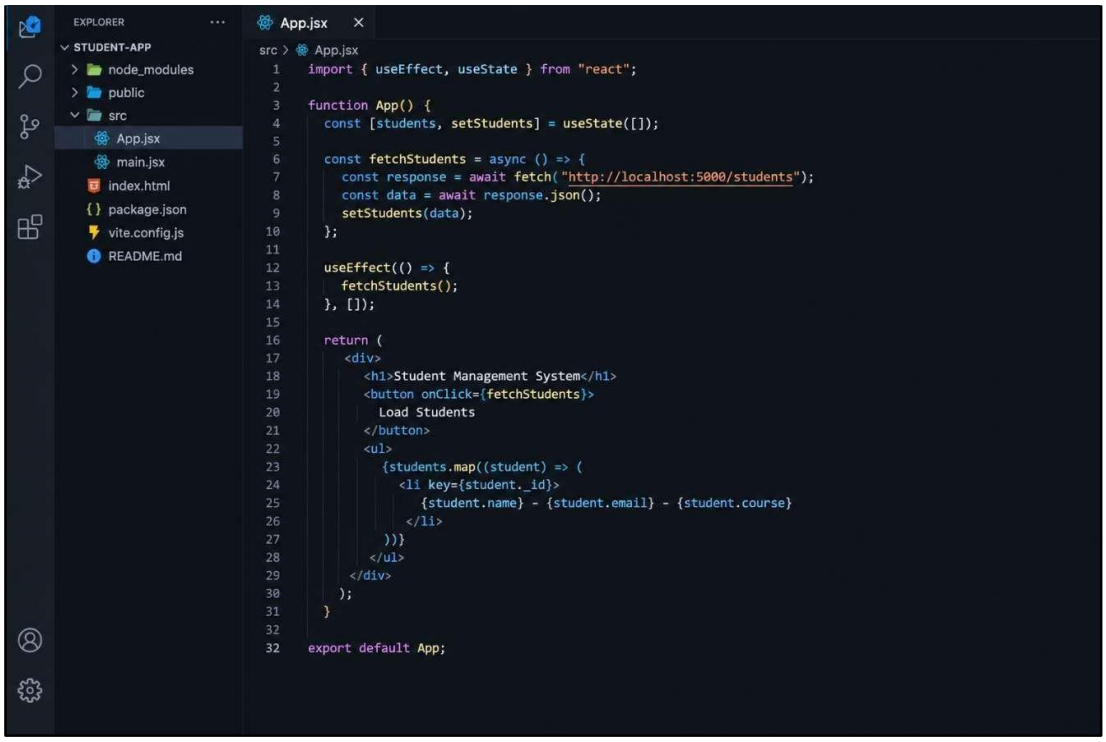
http://localhost:5173

Step 4 — App.jsx Open:

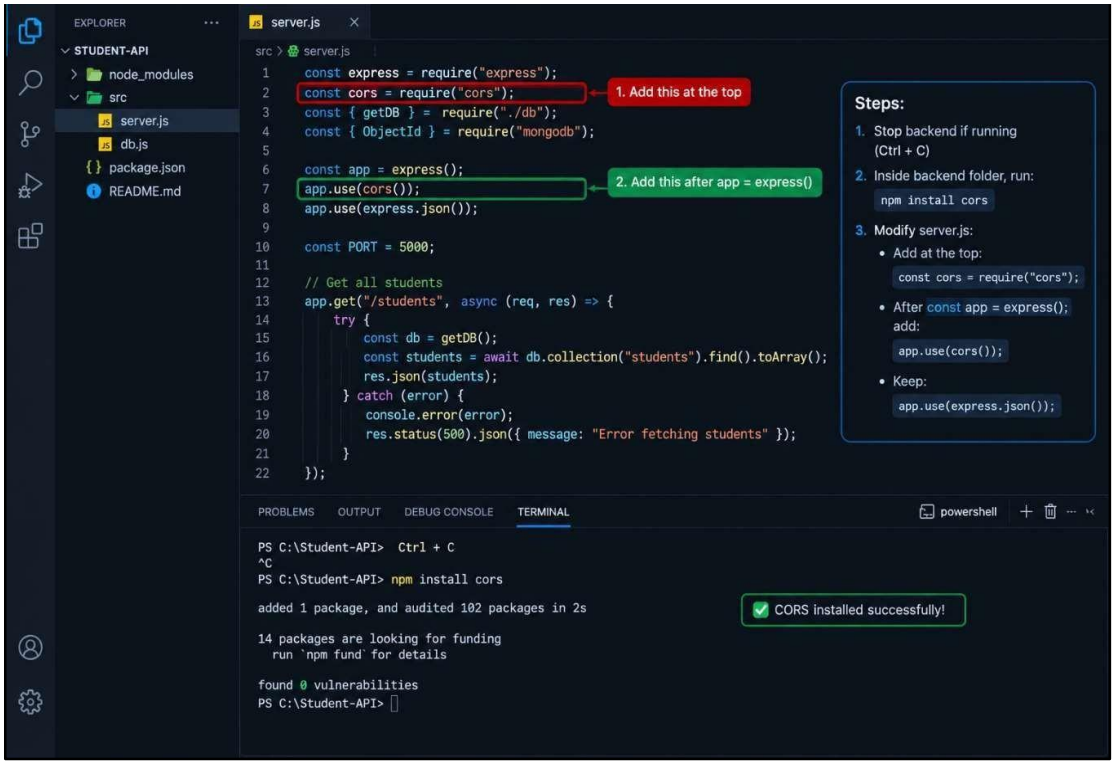
frontend/src/App.jsx

Delete everything.

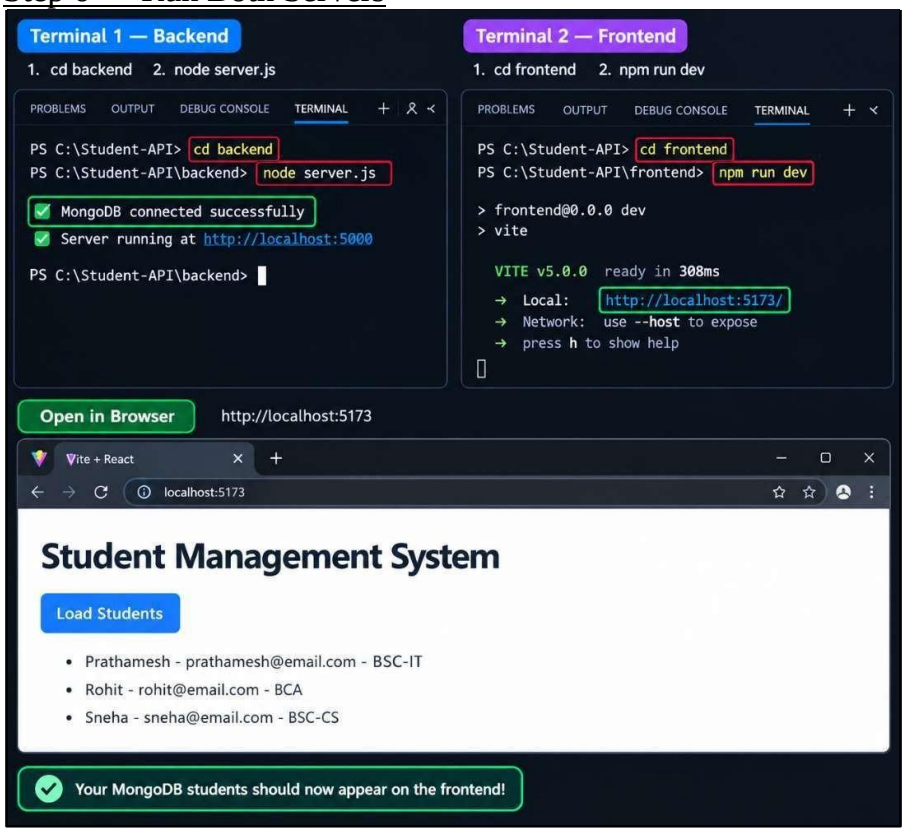
Put:



Step 5 — Add CORS



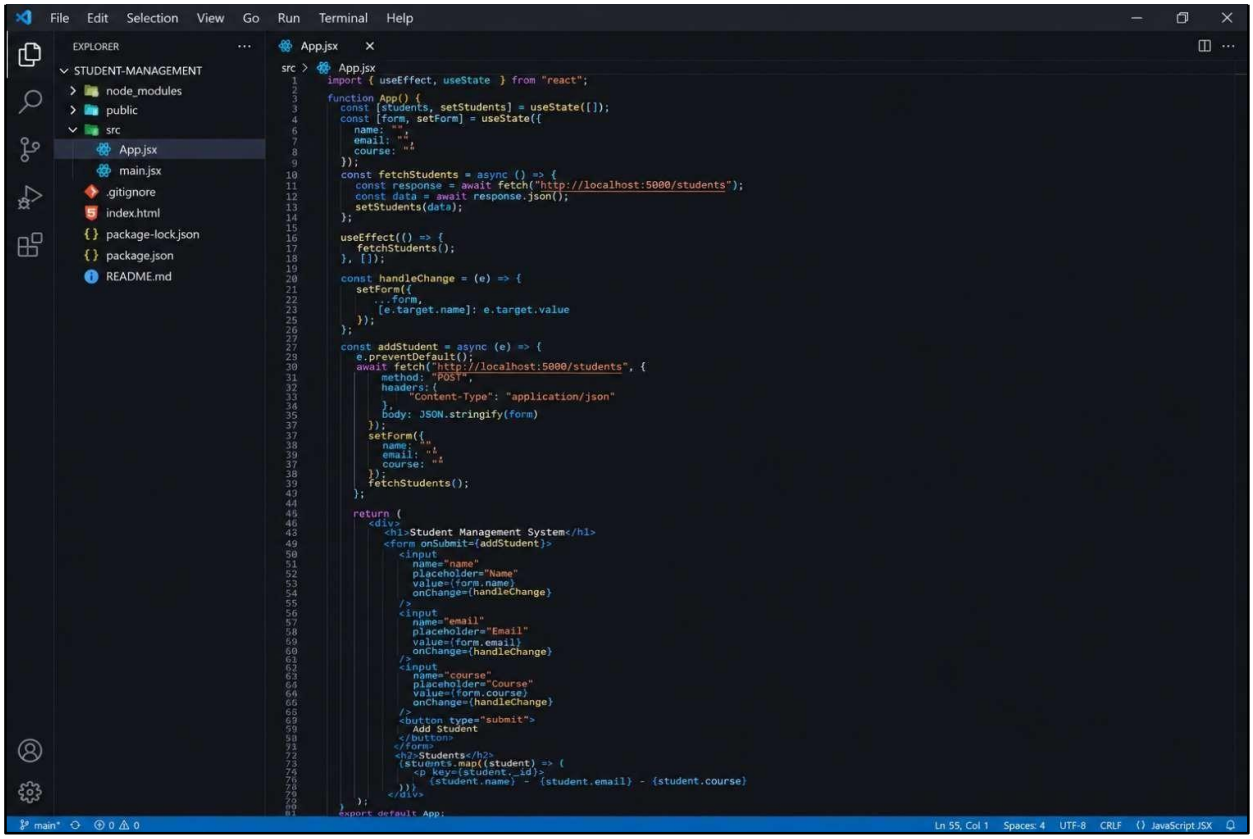
Step 6 — Run Both Servers



POST From React

Now create a form

Code : App.jsx



:Now you can

1. React Form
2. ↓
3. POST
4. ↓
5. Express
6. ↓
7. MongoDB
8. ↓
9. GET
10. ↓
11. React